Exp1:

server:

```
#include<stdio.h>
#include<stdlib.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<unistd.h>
#include<string.h>

int main()
{
        int server_sock;
        int n;
        server_sock=socket(AF_INET,SOCK_STREAM,0);
        struct sockaddr_in server_addr;
        server_addr.sin_family=AF_INET;
        server_addr.sin_port=htons(7249);
        server_addr.sin_addr.s_addr=INADDR_ANY;
        bind(server_sock,(struct sockaddr*)&server_addr,sizeof(server_addr));
        listen(server_sock,5);
        int client_sock;
        client_sock=accept(server_sock,NULL,NULL);
        int upper=1,lower=1,diagonal=1;
        recv(client_sock,&n,sizeof(n),0);
        int matrix[n][n];
        recv(client_sock,matrix,sizeof(matrix),0);
        for(int i=0;i<n;i++)
        {
         for(int j=0;j<n;j++)
         {
                if(i>j && matrix[i][j]!=0)
                {
                 upper=0;
                }
                if(i<j && matrix[i][j]!=0)
                {
                 lower=0;
                }
                if(i!=j && matrix[i][j]!=0)
                {
                 diagonal=0;
```

```
                }
            }
        }

        char result[50];
        if(diagonal)
          strcpy(result,"Diagonal Matrix");
        else if(upper)
          strcpy(result,"Upper Triangle Matrix");
        else if(lower)
          strcpy(result,"Lower Triangle Matrix");
        else
          strcpy(result,"None");
            send(client_sock,result,sizeof(result),0);
            close(client_sock);
            close(server_sock);
            return(0);
}


client:
#include<stdio.h>
#include<stdlib.h>
#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include<unistd.h>
#include<time.h>

int main()
{
            int sock,n;
            char result[30];
            sock=socket(AF_INET,SOCK_STREAM,0);
            struct sockaddr_in server_addr;
            server_addr.sin_family=AF_INET;
            server_addr.sin_port=htons(7249);
            server_addr.sin_addr.s_addr=INADDR_ANY;
            int conn_status=connect(sock,(struct sockaddr*)&server_addr,sizeof(server_addr));
            if(conn_status==-1)
                        printf("Error in making a connection\n");
            printf("Enter order of matrix:");
            scanf("%d",&n);
            int matrix[n][n];
```

```
        //srand(time(NULL));
        printf("Enter the Matrix:\n");
        for( int i=0;i<n;i++)
        {
        for(int j=0;j<n;j++)
        {
        scanf("%d",&matrix[i][j]);
        }
        }
        printf("Matrix:\n");
        for( int i=0;i<n;i++)
        {
        for(int j=0;j<n;j++)
        {
        printf("%d\t",matrix[i][j]);
        }
        printf("\n");
        }
        send(sock,&n,sizeof(n),0);
        send(sock,matrix,sizeof(matrix),0);
        recv(sock,result,sizeof(result),0);
        printf("server's answer: %s \n",result);
        close(sock);
        return(0);
}
```

Output:

```
//client
Enter order of matrix:3
Enter the Matrix:
5
4
5
0
8
3
0
0
4
Matrix:
5       4       5
0       8       3
```

0 0 4
server's answer: Upper Triangle Matrix

Exp 2:

server:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <arpa/inet.h>
#include <unistd.h>

#define PORT 8080

void translate(char str[])
{
    struct
    {
        char shortf[20];
        char full[50];
    } dict[] =
    {
        {"tbh","to be honest"},
        {"sus","suspicious"},
        {"ig","I guess"},
        {"tbf","to be fair"},
        {"atm","at the moment"},
        {"irl","in real life"},
        {"lol","laughing out loud"},
        {"asap","as soon as possible"},
        {"omg","oh my God"},
        {"ttyl","talk to you later"},
        {"idk","I don't know"},
        {"nvm","never mind"},
        {"idc","I don't care"}
    };

    int size = sizeof(dict) / sizeof(dict[0]);

    char result[1024] = "";
    char temp[1024];
    strcpy(temp, str);
```

```
    char *token = strtok(temp, " ");
    while (token != NULL)
    {
        int found = 0;

        for (int i = 0; i < size; i++)
        {
            if (strcmp(token, dict[i].shortf) == 0)
            {
                strcat(result, dict[i].full);
                strcat(result, " ");
                found = 1;
                break;
            }
        }

        if (!found)
        {
            strcat(result, token);
            strcat(result, " ");
        }

        token = strtok(NULL, " ");
    }

    strcpy(str, result);
}

int main()
{
    int server_sock;
    char buffer[1024];

    struct sockaddr_in serverAddr, clientAddr;
    socklen_t len;

    server_sock = socket(AF_INET, SOCK_DGRAM, 0);

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = INADDR_ANY;

    bind(server_sock, (struct sockaddr *)&serverAddr, sizeof(serverAddr));
```

```
    printf("UDP Server Running...\n");

    len = sizeof(clientAddr);

    recvfrom(server_sock, buffer, sizeof(buffer), 0,
            (struct sockaddr *)&clientAddr, &len);

    printf("Received: %s\n", buffer);

    translate(buffer);

    sendto(server_sock, buffer, strlen(buffer) + 1, 0,
           (struct sockaddr *)&clientAddr, len);

    close(server_sock);

    return 0;
}
```

client:

```
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>

#define PORT 8080

int main()
{
int sockfd;
char buffer[1024];

struct sockaddr_in serverAddr;

sockfd = socket(AF_INET, SOCK_DGRAM, 0);

serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(PORT);
serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

printf("Enter Sentence:\n");
fgets(buffer,sizeof(buffer),stdin);
```

```
sendto(sockfd,buffer,strlen(buffer)+1,0,
       (struct sockaddr*)&serverAddr,sizeof(serverAddr));

recvfrom(sockfd,buffer,sizeof(buffer),0,NULL,NULL);

printf("\nTranslated Sentence:\n%s\n",buffer);

close(sockfd);

return 0;
}
```

output:
//server
UDP Server Running...
Received: omg

//client
Enter Sentence:
omg
Translated Sentence:
oh my God

Exp 3:

server:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <pthread.h>
#define MAX_CLIENTS 10

int clients[MAX_CLIENTS];
int clientCount = 0;
pthread_mutex_t lock;

void broadcast(char message[], int sender)
{
   pthread_mutex_lock(&lock);
```

```
    for(int i = 0; i < clientCount; i++)
    {
        if(clients[i] != sender)
        {
            send(clients[i], message, strlen(message), 0);
        }
    }

    pthread_mutex_unlock(&lock);
}

void *handleClient(void *arg)
{
    int clientSocket = *(int *)arg;
    char buffer[1024];

    while(1)
    {
        int bytes = recv(clientSocket,buffer,sizeof(buffer), 0);
        if(bytes <= 0)
            break;
            buffer[bytes] = '\0';
            printf("Client %d: %s", clientSocket, buffer);
            broadcast(buffer, clientSocket);
    }
    close(clientSocket);

    pthread_mutex_lock(&lock);

    for(int i=0;i<clientCount;i++)
    {
        if(clients[i]==clientSocket)
        {
            for(int j=i;j<clientCount-1;j++)
            {
                clients[j]=clients[j+1];
            }

            clientCount--;
            break;
        }
    }

    pthread_mutex_unlock(&lock);
```

```
    free(arg);
    pthread_exit(NULL);
}

int main()
{
    int serverSocket;
    struct sockaddr_in serverAddr, clientAddr;
    socklen_t addrSize;

    pthread_mutex_init(&lock, NULL);

    serverSocket = socket(AF_INET, SOCK_STREAM, 0);

    if(serverSocket < 0)
    {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
    }

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(5746);
    serverAddr.sin_addr.s_addr = INADDR_ANY;

    if(bind(serverSocket, (struct sockaddr *)&serverAddr,
            sizeof(serverAddr)) < 0)
    {
        perror("Bind failed");
        close(serverSocket);
        exit(EXIT_FAILURE);
    }

    if(listen(serverSocket, MAX_CLIENTS) < 0)
    {
        perror("Listen failed");
        close(serverSocket);
        exit(EXIT_FAILURE);
    }

    printf("Chat Server Running...\n");

    while(1)
    {
```

```
        addrSize = sizeof(clientAddr);

        int *clientSocket = malloc(sizeof(int));

        *clientSocket = accept(serverSocket,(struct sockaddr *)&clientAddr,&addrSize);

        if(*clientSocket < 0)
        {
            perror("Accept failed");
            free(clientSocket);
            continue;
        }

        pthread_mutex_lock(&lock);

        if(clientCount < MAX_CLIENTS)
        {
            clients[clientCount++] = *clientSocket;
        }
        else
        {
            printf("Maximum clients reached.\n");
            close(*clientSocket);
            free(clientSocket);
            pthread_mutex_unlock(&lock);
            continue;
        }

        pthread_mutex_unlock(&lock);

        pthread_t tid;

        pthread_create(&tid, NULL, handleClient, clientSocket);
        pthread_detach(tid);
    }

    close(serverSocket);
    pthread_mutex_destroy(&lock);

    return 0;
}


client:
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <pthread.h>

#define PORT 4213
#define BUFFER_SIZE 1024

int sockfd;

void *receiveMessage(void *arg)
{
    char buffer[BUFFER_SIZE];

    while(1)
    {
        int bytes = recv(sockfd, buffer, BUFFER_SIZE - 1, 0);

        if(bytes <= 0)
        {
            printf("\nDisconnected from server.\n");
            break;
        }

        buffer[bytes] = '\0';

        printf("\nMessage: %s", buffer);
    }

    pthread_exit(NULL);
}

int main()
{
    struct sockaddr_in serverAddr;

    sockfd = socket(AF_INET, SOCK_STREAM, 0);

    if(sockfd < 0)
    {
        perror("Socket creation failed");
        exit(EXIT_FAILURE);
```

```
    }

    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    if(connect(sockfd,
            (struct sockaddr *)&serverAddr,
            sizeof(serverAddr)) < 0)
    {
        perror("Connection failed");
        close(sockfd);
        exit(EXIT_FAILURE);
    }

    printf("Connected to Chat Server.\n");

    pthread_t tid;
    pthread_create(&tid, NULL, receiveMessage, NULL);

    char buffer[BUFFER_SIZE];

    while(1)
    {
        if(fgets(buffer, sizeof(buffer), stdin) == NULL)
            break;

        send(sockfd, buffer, strlen(buffer), 0);
    }

    close(sockfd);
    return 0;
}

output:
 //server
Chat Server Running...

//client
Connected to Chat Server.
hii
Message: hiiii
omg
Message: helloo
```

Exp 4:

server:

```
#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<arpa/inet.h>
#include<unistd.h>
#include<time.h>

int main()
{
        int server_sock;
        char buffer[1024],now[100];
        time_t t;
        struct sockaddr_in serverAddr, clientAddr;
        server_sock = socket(AF_INET, SOCK_DGRAM, 0);
        serverAddr.sin_family = AF_INET;
        serverAddr.sin_port = htons(7564);
        serverAddr.sin_addr.s_addr = INADDR_ANY;
        bind(server_sock, (struct sockaddr *)&serverAddr, sizeof(serverAddr));
        printf("UDP Server Running...\n");
        socklen_t len=sizeof(clientAddr);
        while(1)
        {
                recvfrom(server_sock, buffer, sizeof(buffer), 0,(struct sockaddr
*)&clientAddr,&len);
        time(&t);
        strcpy(now,ctime(&t));
        sendto(server_sock,now,sizeof(now),0,(struct sockaddr*)&clientAddr,len);
      }
        close(server_sock);
        return 0;
}
```

client:

```
#include <stdio.h>
#include <string.h>
#include <arpa/inet.h>
#include <unistd.h>
```

```
int main()
{
int sockfd;
char buffer[1024];

struct sockaddr_in serverAddr;

sockfd = socket(AF_INET, SOCK_DGRAM, 0);

serverAddr.sin_family = AF_INET;
serverAddr.sin_port = htons(7564);
serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");

strcpy(buffer,"TIME REQUEST");
sendto(sockfd,buffer,strlen(buffer)+1,0,(struct sockaddr*)&serverAddr,sizeof(serverAddr));
recvfrom(sockfd,buffer,sizeof(buffer),0,NULL,NULL);

printf("\nCurrent System Time:\n%s\n",buffer);

close(sockfd);

return 0;
}
```

output:
//server
UDP Server Running...
//client
Current System Time:
Wed Sep 9 09:53:56 2026

Exp 5:

server:

```
#include<stdio.h>
#include<stdlib.h>
```

```
#include<unistd.h>
#include<string.h>
#include<netinet/in.h>
#include<sys/wait.h>

#define PORT 7264
#define BUFFER_SIZE 1024

int main()
{
	int server_sock,client_sock;
	struct sockaddr_in server_addr,client_addr;
	char filename[BUFFER_SIZE],buffer[BUFFER_SIZE];
	server_sock=socket(AF_INET,SOCK_STREAM,0);
	server_addr.sin_family=AF_INET;
	server_addr.sin_port=htons(PORT);
	server_addr.sin_addr.s_addr=INADDR_ANY;
	bind(server_sock,(struct sockaddr*)&server_addr,sizeof(server_addr));
	listen(server_sock,10);
	printf("File server running on PORT %d\n",PORT);
	while(1)
	{
		int client_len=sizeof(client_addr);
		client_sock=accept(server_sock,(struct sockaddr*)&client_addr,&client_len);
		pid_t pid=fork();
		if(pid<0)
		{
			perror("Fork failed\n");
			exit(0);
		}
		else if(pid==0)
		{
			close(server_sock);
			memset(filename,0,BUFFER_SIZE);
			int n=recv(client_sock,filename,sizeof(filename),0);
			if(n<0)
				close(client_sock);
			filename[n]='\0';
			FILE *fp=fopen(filename,"r");
			sprintf(buffer,"Server Process ID: %d\n",getpid());
			send(client_sock,buffer,strlen(buffer),0);
			if(fp==NULL)
			{
				sprintf(buffer,"File %s does not exists",filename);
```

```
                        send(client_sock,buffer,strlen(buffer),0);
                }
                else
                {
                        sprintf(buffer,"------File Contents---------\n");
                        send(client_sock,buffer,strlen(buffer),0);
                        while(fgets(buffer,BUFFER_SIZE-1,fp))
                        {
                                send(client_sock,buffer,strlen(buffer),0);
                        }
                        /*memset(buffer,0,BUFFER_SIZE);
                        fgets(buffer,BUFFER_SIZE-1,fp);
                        send(server_sock,buffer,strlen(buffer),0);*/
                        fclose(fp);
                }
                close(client_sock);
                exit(0);
        }
        else
        {
                close(client_sock);
                waitpid(-1,NULL,WNOHANG);
        }
    }
    close(server_sock);
    return 0;
}

client:
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<string.h>
#include<netinet/in.h>
#include<sys/wait.h>
#define PORT 7264
#define BUFFER_SIZE 1024

int main()
{
    int client_sock;
    struct sockaddr_in server_addr;
    char buffer[BUFFER_SIZE],filename[BUFFER_SIZE];
    client_sock=socket(AF_INET,SOCK_STREAM,0);
```

```
        server_addr.sin_family=AF_INET;
        server_addr.sin_port=htons(PORT);
        server_addr.sin_addr.s_addr=INADDR_ANY;
        connect(client_sock,(struct sockaddr*)&server_addr,sizeof(server_addr));
        printf("Enter the filename:");
        scanf("%s",filename);
        //printf("%s",filename);
        send(client_sock,filename,strlen(filename),0);
        printf("Filename is sent\n");
        recv(client_sock,buffer,strlen(buffer),0);
        printf("%s",buffer);
        int n;
        /*while((n=recv(client_sock,buffer,BUFFER_SIZE,0))>0)
        {
                buffer[n]='\0';
                printf("%s\n",buffer);
        }*/
        n=recv(client_sock,buffer,BUFFER_SIZE,0);
        buffer[n]='\0';
        printf("%s\n",buffer);
        close(client_sock);
        return 0;
}
```

output:

```
//server
File server running on PORT 7264

//client
Enter the filename:exp_file.txt
Filename is sent
Server Process ID: 289618
------File Contents---------
Hello
```